

Lab Assignment 5

Christos Strydis
Rajendra Bishnoi
Amirreza Movahedin

2026

Contents

1	Introduction	2
2	CPU and GPU	2
3	Tensor Processing Unit (TPU)	2
4	Field-Programmable Gate Array (FPGA)	4
5	Comparison	5

Submission guidelines:

- Hand in the lab report in pdf format with the following name:
Lab5_Firstname_Lastname_Studentnumber (for example: Lab5_Pieter_deBoer_123456)

1 Introduction

In the previous lab, we looked into the basic concepts of AI and trained some models. In this lab, we will explore deploying an AI model on a variety of hardware backends, including CPU, GPU, Tensor Processing Unit (TPU), and Field-Programmable Gate Array (FPGA). For each of these backends, we are gonna look into their latency and throughput, in addition to power and energy consumptions. The AI model that we are using for this lab, for the sake of having the least amount of compatibility issues, is a 5-layer MLP with 16 input, 64 hidden, and 10 output neurons and ReLu activation.

2 CPU and GPU

To benchmark the performance of CPU and GPU when executing our AI model, we are loading the stored model in JAX and run it on different backends. Open the `cpu_gpu.py` script. Here you can find the JAX-based code for benchmarking our model on CPU or GPU, depending on the chosen backend.

Exercise 5.1 (10 points)

Report the latency CPU and GPU when running a single inference of the model. Additionally, report the power consumption of the GPU when running a single inference. You can use `nvidia-smi` in addition to the `watch` command to monitor the power consumption of the GPU. You can run a large number of inferences back-to-back to see a more sustained power consumption value while monitoring. Assuming the CPU consumes 100 Watts of power when running an inference, what is the energy consumption of these devices when performing a single inference? How do these backends compare when it comes to single-inference latency and energy consumption? (Note that it is possible to measure the energy consumption of certain CPU models using Running Average Power Limit feature, without having to have physical access to them. However, this is not possible on virtualized CPUs such as the ones on AWS instances.)

Exercise 5.2 (10 points)

Let's measure the throughput of these backends. To do this, we are going to run batched inference on the devices. How would you change the code to run batched inference? Find the maximum throughput (in terms of inferences/sec) of both CPU and GPU by sweeping the batch size. Also, measure and report the power, energy, and throughput per Watt of the GPU at its maximum throughput. How do CPU and GPU compare in terms of throughput? What are your takeaways from the results you have seen in this and previous exercise?

3 Tensor Processing Unit (TPU)

As you have seen in the lectures, TPU is a specialized hardware accelerator developed by Google to efficiently perform machine learning and deep learning computations. Unlike general-purpose CPUs or GPUs, TPUs are optimized for large-scale matrix operations commonly used in neural networks, such as training and inference in models built with frameworks like TensorFlow and JAX. Access to physical TPUs is generally limited, typically available through cloud-based research programs or restricted free-tier access on platforms such as Google Colab. Because direct access to production-scale TPU hardware is not consistently available, we are using an industry-grade, open-source simulator to emulate TPU behavior called NeuSim [4, 5, 6]. NeuSim is a general Neural Processing Unit (NPU) simulation framework, allowing for configuring different devices and benchmarking them with different machine learning workloads. The general NPU architecture NeuSim assumes is shown in Figure 1. Google's TPU also follows the same architecture. Additionally, NeuSim has been verified against actual TPU hardware of different generations, making it an accurate representation of Google's processor.

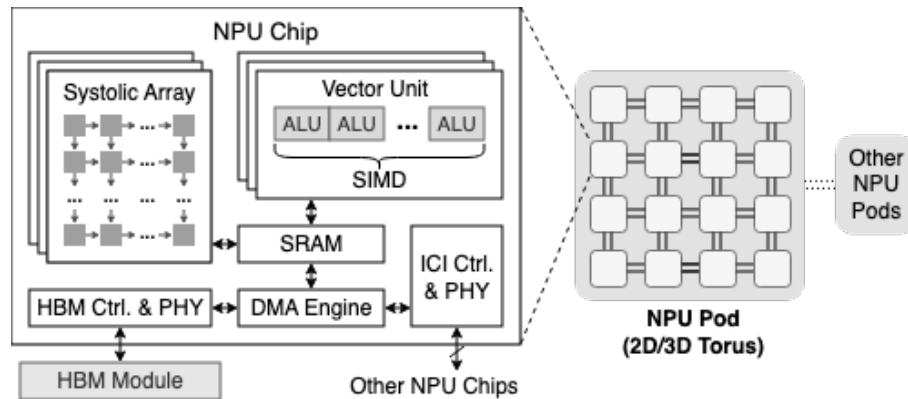


Figure 1: General NPU architecture assumed in the NeuSim framework

Exercise 5.3 (5 points)

What are the roles of systolic array and vector unit in the TPU architecture?

Exercise 5.4 (17.5 points)

Open the `tpu.py` script. For an AI model to benchmark in the simulator, NeuSim requires a model configuration class, which holds the configuration parameters of a model, and an operator generator, which defines the layers and operations conducted in a model. We defined these two classes for an MLP model, namely `MLPConfig` and `MLPOpeGenerator` in the `tpu.py` script. Then, we load the NPU, in this case TPUv2, and system configurations. In the next step, we define `model.cfg` which is the configurations of the MLP we want to run. The next lines prepares and instantiates the required classes we need to simulate the hardware. The simulation results, divided by each operation/layer, will be written to the `./results/mlp-inference-v2.csv` file.

Note that to run this script, you need to use the `tpu-env` Python environment. An easy way to do that is to run the script as follows

```
tpu-env/bin/python tpu.py
```

1. Run the code for a single inference and inspect the output of the simulation. Report the total execution time, power, and energy consumption. (5 points)
2. Sweep the batch size of the inference to find the maximum throughput of TPUv2 when running this model. Report your findings. (5 points)
3. Compare latency and throughput of TPUv2 with CPU and GPU when running our MLP. Also compare the power and energy consumption of TPUv2 and GPU both for single and batched (high-throughput) inference. How does TPUv2's throughput per Watt compare to that of GPU? (7.5 points)

Exercise 5.5 (12.5 points)

1. When examining the results, we can see that both systolic array (MXU) and vector unit (VPU) are being used. What operations do you think each of the units are executing? (5 points)
2. When running single inferences, we can see that the execution is memory-bound. However, running batched inference (for example batch size of 1024), the model execution becomes compute-bound. How do you explain this difference? (5 points)
3. Run a single inference when `use_vu_for_small_matmul` in the `tpuv2.json` file is set to true or false. Compare and explain the difference in the output result between these two modes. (2.5 points)

4 Field-Programmable Gate Array (FPGA)

A Field-Programmable Gate Array (FPGA) is a reconfigurable hardware device that allows users to implement custom digital circuits optimized for specific applications. FPGAs can be programmed at the hardware level to achieve highly parallel, low-latency, and energy-efficient execution, making them well suited for real-time AI inference and edge computing applications.

In this lab, we are going to use hls4ml [1, 2, 3] for AI inference on FPGA hardware. Originally developed for real-time machine learning applications at CERN, hls4ml is an open-source framework that turns high-level machine learning models into deployable FPGA kernels. This is done by converting trained machine learning models from frameworks such as TensorFlow, Keras, and PyTorch into synthesizable High-Level Synthesis (HLS) code that can be compiled for FPGA hardware. This enables efficient hardware acceleration of AI inference while abstracting much of the low-level hardware description complexity, in addition to enabling fast exploration of different models and FPGA implementations. The framework also supports optimization techniques such as quantization, pipelining, and parallelization to reduce latency and FPGA resource usage.

Exercise 5.6 (5 points)

What general architecture does hls4ml follow and what is the reuse factor in this architecture? (Hint: look into the documentation of hls4ml)

Exercise 5.7 (10 points)

Open the `fpga.py` script and examine the code. We first load the saved MLP (in the form of a Keras model). Then, we configure the model in hls4ml by setting its precision and reuse factors. These parameters can also be configured layer-wise. After that, we convert the model into an HLS project. This step creates the `./mlp_hls/` directory, and puts the generated HLS code under `firmware` directory. In the next step, we synthesize the generated HLS project and run a C/RTL co-simulation.

1. Run the code and check the HLS synthesis report found under `./mlp_hls/myproject_prj/solution1/syn/report`. What is the latency of the design in ns? What is the utilization (in terms of number of resources used) of the FPGA for this design? Note that we are not going to run our design on an actual FPGA, but since the AI workload is highly deterministic, relying on the synthesis report is accurate enough. (5 points)
2. What is the end-to-end latency (in ns) of running N back-to-back inferences with this design? Additionally, estimate the throughput of the design in inferences/sec. (5 points)

Exercise 5.8 (15 points)

1. Decrease the frequency of the design to 100 MHz by setting the clock period to 10 ns and report the latency in ns. How do you explain the results? (5 points)
2. At 200 MHz, change the quantization precision of the model to Q2.2 and report the latency and utilization. How do you explain the results? What is the downside of more aggressive quantization? (5 points)
3. At default precision and 200 MHz, change the reuse factor to 4, what do you expect to happen? Verify your prediction with the synthesis report. In this case, what is the end-to-end latency (in ns) and throughput (in inferences/s) of running N back-to-back inferences? (5 points)

Exercise 5.9 (5 points)

In order to get a power estimate of our design, we have to perform a Vivado synthesis and get a device-specific power consumption report. The power report that Vivado provides after synthesis is merely an estimate, but it would be accurate enough for our purposes. In order to do so, run the hls4ml script, and Then, change the `mlp_hls/vivado_synth.tcl` file as follows:

```
set tcldir [file dirname [info script]]
source [file join $tcldir project.tcl]

add_files ${project_name}_prj/solution1/syn/verilog
synth_design -top ${project_name} -part $part

create_clock -period 5 [get_ports ap_clk]

opt_design -retarget -propconst -sweep -bram_power_opt -shift_register_opt

report_utilization -file vivado_synth.rpt
report_power -file vivado_power.rpt
```

Then, perform the the following commands in your terminal. This will take some time. After the command is done, you can find the power report in `vivado_power.rpt`.

```
cd mlp_hls
vivado -mode batch -source vivado_synth.tcl
```

Keep quantization precision and reuse factor at their default values and report the power consumption value of the design for 100 and 200 MHz clock frequencies (remember to change the period value in the TCL file as well). What are the energy consumption (for single inference) and throughput per Watt (for N back-to-back inferences) of these designs? How do they compare, and how do you explain these results?

5 Comparison

Exercise 5.10 (10 points)

Provide an analysis of how you would go about choosing the right platform for different AI training and inference scenarios. You can explore training vs inference requirement, generality, size and complexity of the models, and inference requirements.

References

Javier Duarte et al. Fast inference of deep neural networks in FPGAs for particle physics. *JINST*, 13(07):P07027, 2018.

FastML Team. hls4ml, 2025.

Jan-Frederik Schulte et al. hls4ml: A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware. 12 2025.

Yuqi Xue, Yiqi Liu, and Jian Huang. System virtualization for neural processing units. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems*, HotOS'23, Providence, RI, USA, 2023.

Yuqi Xue, Yiqi Liu, Lifeng Nai, and Jian Huang. V10: Hardware-assisted npu multi-tenancy for improved resource utilization and fairness. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA'23, Orlando, FL, USA, 2023.

Yuqi Xue, Yiqi Liu, Lifeng Nai, and Jian Huang. Hardware-assisted virtualization of neural processing units for cloud platforms. In *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture*, MICRO'24, Austin, TX, USA, 2024.